EXERCISE SET / SIX

Recursion, DFS and Fast Power

洛谷六题习题解析

数的划分 · 外星密码 · 组合的输出 · 迷宫 · 幂次方 · 快速幂

每道题用三页讲透：这一页看清题面、下一页拆开思路、最后一页逐行读代码。

六题围绕递归与搜索两条主线，最后另附三页快速幂专题讲解。

题目来源

洛谷 luogu.com.cn ｜ A — F 六题 ｜ 每题 3 页 + 快速幂专题 3 页

主讲 · 信息教研组 ｜ 授课班级 · 中预 2026 级 B 班

六题地图

六道题按「搜索 → 递归 → 分治」排列：A、C、D 是深度优先搜索的三种典型用法，B、E 是递归展开，F 及其后的专题讲快速幂。

A	P1025	数的划分	DFS • 剪枝	不降序列去掉重复	第 03 – 05 页
B	P1928	外星密码	递归 • 模拟	遇左括号进一层	第 06 – 08 页
C	P1157	组合的输出	DFS • 枚举	从上一个数往后选	第 09 – 11 页
D	P1605	迷宫	DFS • 回溯	走时标记回来还原	第 12 – 14 页
E	P1010	幂次方	递归 • 分治	指数也要展开	第 15 – 17 页
F	P1226	快速幂	分治 • 二进制	指数按二进制拆	第 18 – 20 页

做题顺序建议

先做 C、D 两题热身：组合枚举与迷宫回溯是 DFS 的两个基本盘；再做 B、E 两道递归展开题；最后做带剪枝的 A 与模板题 F，并连读第 21 至 23 页的快速幂专题。

A 题 • 读懂题面

P1025 数的划分

DFS · 剪枝

把整数 n 分成 k 份，每份不能为空，相同的几种排法只算一种（不考虑顺序）。问共有多少种不同的分法。

输入格式

一行两个整数 n 与 k：
要分的整数、以及份数。
6 小于 n 且 n ≤ 200，2 ≤ k ≤ 6。

输出格式

一个整数，即不同分法的总数。
同一组数换一种排列顺序，仍然只算同一种分法。

数据范围

n 最大 200，k 最大 6。
方案数在几百量级，int 足够。
★ k 很小，搜索加剪枝轻松通过。

样例

```
7 3
```

```
4
```

四种分法为 1,1,5；1,2,4；1,3,3；2,2,3。
注意 1,1,5 与 5,1,1 与 1,5,1 是同一种，只数一次。
关键：搜索时强制每个数不小于前一个数。

题意建模的一句话

这是「把 n 个球放进 k 个无序盒子」的计数问题。用 DFS 逐份填数，只要强制「后一份不小于前一份」，就天然去掉了重复方案。

A 题 · 思路拆解

直接枚举所有 k 元组会数出大量重复。唯一的技巧是给搜索定一个「不下降」的规则，让每种分法只被枚举一次。

第一步：按不降序列填数

第 1 份从 1 开始试；之后每一份都从前一份的值开始试。这样 1,5,1 之类的乱序方案根本不会被生成，天然去重。

第二步：递归三要素

参数记三样：上一份填了多少 last、当前总和 sum、已填份数 c。填满 k 份时若 sum 恰好等于 n，方案数加一。

第三步：剪枝上限

还剩 k−c 份，每份至少是 i，所以只有 sum + i × (k−c) ≤ n 的 i 才值得试。写成循环条件，枚举量大幅缩小。

第四步：边界与答案

第一份从 1 起而不是 0 起（每份非空）；cnt 是全局计数器，递归结束后直接输出。

一个可以顺带记住的 DP 视角

设 f[n][k] 为 n 分 k 份的方案数，则 f[n][k] = f[n−1][k−1] + f[n−k][k]：有 1 的分法拆掉一个 1，没有 1 的分法每份减 1。

A 题 · 参考代码


```
#include <cstdio>
using namespace std;

int n, k, cnt = 0;

void dfs(int last, int sum, int c) {  // 上一份 last，总和 sum，已填 c 份
    if (c == k) {                     // k 份都填完
        if (sum == n) cnt++;          // 恰好分完才计数
        return;
    }
    for (int i = last; sum + i * (k - c) <= n; i++)
        dfs(i, sum + i, c + 1);       // 本份取 i，下一份从 i 起仍不降
}

int main() {
    scanf("%d %d", &n, &k);
    dfs(1, 0, 0);                     // 第一份从 1 开始试
    printf("%d
", cnt);
    return 0;
}
```

去重全靠起点 last

内层 i 从 last 开始而不是从 1 开始，保证序列不下降。这一处是本题与普通枚举的唯一区别，也是最容易被漏掉的一行。

剪枝写在循环条件里

sum + i 乘 (k−c) 不超过 n 才继续：剩下的份数每份至少 i，一旦超出就直接停，后面的 i 更大更不必试。

计数时机在叶子

c 等于 k 时才能下结论，中间层永远不计数。sum 小于 n 的「半成品」会随循环自然结束而被放弃。

复杂度

有剪枝后状态数远小于理论枚举量，n ≤ 200、k ≤ 6 时毫秒级出解。若把 k 放大，就要换上面提到的 DP 写法。

B 题 · 读懂题面

P1928

外星密码

递归 · 模拟

外星人的密码做了压缩：连续相同的子串 X 被写成 [DX]，D 是重复次数。压缩可以嵌套。现在给出压缩串，要求还原原文。

输入格式

一行一个字符串，即压缩后的密码。
只含数字、大写字母、[和]。
[后的 D 是 1 到 99 的整数。

输出格式

一行一个字符串，即解压后的原文。
嵌套压缩要一层层展开到最里。

数据范围

解压后长度 ≤ 20000。
最多十重压缩。
★ 重复次数最多两位数，读到非数字即结束。

样例

```
AC[3FUN]
```

```
ACFUNFUNFUN
```

[3FUN] 表示 FUN 重复 3 遍，前面还有 AC，所以结果如右。
再看嵌套：[2[2CB]] 先展开最里层得 CBCB，
再把 CBCB 重复 2 遍，得 CBCBCBCB。

题意建模的一句话

一层方括号就是一层函数调用：读入次数、递归展开括号里的内容、重复拼接。字符串天然按「层」组织，正对递归的胃口。

B 题 · 思路拆解

整个解压过程只有一个动作：从左到右逐字符扫描。难点在「读到 [时要交出控制权」，让递归去处理括号里的整段。

第一步：逐字符扫描，分三类

普通字母直接拼进当前结果；左括号 [开启一段压缩；右括号] 表示当前层结束，把结果交还上一层。

第二步：先读重复次数

D 可能是两位数，遇到 [后要连续读数字，直到读到非数字字符为止，再把这个数记下来。

第三步：递归展开括号内

对括号里的内容调用同一个函数，得到完全展开的子串；随后把子串重复 D 次拼进当前层的结果。

第四步：用全局指针接力

递归返回时，扫描指针恰好停在对应的] 之后。上一层从那里无缝继续，这正是递归分层的意义。

两个常见坑

一是次数 D 忘了读两位数；二是想用 cin 整串读入却不知在哪截断，改用逐字符扫描、指针走到哪算哪，边界自然清晰。

B 题 · 参考代码

信息学竞赛培训 · 洛谷六题习题解析

P1928 · 递归解压

```cpp
#include <iostream>
#include <string>
using namespace std;

string s;
int pos = 0;                    // 全局扫描指针

string unzip() {                // 解压当前层，停在 ] 或串尾
    string res = "";
    while (pos < (int)s.size()) {
        char c = s[pos++];
        if (c == ']') break;        // 当前层结束
        if (c == '[') {             // 压缩段：先读重复次数
            int d = 0;
            while (s[pos] >= '0' && s[pos] <= '9')
                d = d * 10 + (s[pos++] - '0');
            string inner = unzip();  // 递归展开括号内
            while (d--) res += inner;   // 重复 d 次
        } else {
            res += c;               // 普通字符直接拼
        }
    }
    return res;
}

int main() {
    cin >> s;
    cout << unzip() << endl;
    return 0;
}
```

指针只进不退

pos 是全局变量，递归各层共用。每读一个字符就前移一位，返回时刚好落在] 之后，上一层继续扫描不会重读。

两层 while 各司其职

外层 while 走字符；内层 while 只在读次数时用，把连续数字收成一个整数。写混就会出现把字母当次数的怪错误。

结果用 string 拼接

解压后最长 20000，string 直接相加足够快。若手写 char 数组，要小心各层长度与越界。

复杂度

每个输出字符只被写入常数次，总时间与解压后长度同阶，约两万次基本操作，毫无压力。

P1157 组合的输出

DFS · 枚举

从自然数 1 到 n 中任取 r 个数（不分顺序），把所有组合全部输出：每组内从小到大，组与组之间按字典序。

输入格式

一行两个自然数 n 与 r：
候选范围 1 到 n、取出个数 r。
1 小于 n 小于 21，0 ≤ r ≤ n。

输出格式

所有组合各占一行，组内元素从小到大。
★ 每个元素占 3 个字符宽度，
不够要补空格，格式错全盘皆输。

数据范围

n 最大 20，组合数最多 C(20,10)。
即 184756 行输出，量不小。
用 printf 比 cout 更稳更快。

样例

```
5 3
```

```
  1   2   3
  1   2   4
  1   2   5
...
  3   4   5
```

每个数字占 3 位（如左列示意，数字前补空格）。
组合按字典序：先排完 1 开头的全部，再排 2 开头的。
这道题是「枚举什么、按什么顺序」的最佳练习。

题意建模的一句话

DFS 逐位填数：第 t 位只从前一位的值加 1 开始选。这一条起点约束同时解决了三件事——不重复、不乱序、天然字典序。

C 题 · 思路拆解

信息学竞赛培训 · 洛谷六题习题解析

P1157 组合的输出

组合与排列的区别就在于「不计顺序」。让程序只生成递增序列，重复与顺序问题就一起消失了，这是搜索题的标准去重手法。

第一步：确定递归状态

两个参数：t 表示现在填第几位，start 表示这一位最小能取几。a 数组存当前组合，r 位填满即输出。

第二步：起点约束去重

第 t 位从 start 取到 n，下一层传 t+1 与 i+1。序列严格递增，1,3,2 这种乱序永远不会被生成，恰好就是组合的定义。

第三步：输出格式单独处理

用 printf 三位场宽输出每个数。注意题目给的样例里数字前有空格，场宽不够就是格式错误，一个测试点都拿不到。

第四步：递归出口

t 大于 r 时说明 r 位都填好，输出并返回。r 为 0 的边界也要能正常结束，程序才完整。

对照记忆：与 A 题的异同

A 题也用「起点约束」去重，但那里允许相等（1,1,5 合法），所以下一份从 last 起而不是 last 加 1 起；本题数字不能重复，故传 i+1。

C 题 · 参考代码


```cpp
#include <cstdio>
using namespace std;

int n, r, a[25];

void dfs(int t, int start) {        // 填第 t 位，候选从 start 起
    if (t > r) {                    // r 位都选好了
        for (int i = 1; i <= r; i++)
            printf("%3d", a[i]);    // ★ 每个数占 3 个场宽
        printf("
");
        return;
    }
    for (int i = start; i <= n; i++) {
        a[t] = i;
        dfs(t + 1, i + 1);          // 下一位从 i+1 起：递增且不重复
    }
}

int main() {
    scanf("%d %d", &n, &r);
    dfs(1, 1);
    return 0;
}
```

一行两个参数定乾坤

dfs(t, start) 的含义清晰后，主循环只是「从 start 到 n 试一遍」。搜索题的代码量小，功夫全在状态定义上。

场宽 %3d 的含义

输出占 3 个字符位，不足左补空格。数字最多两位，所以永远补 1 至 2 个空格，与题目样例格式一致。

不需要 visited 数组

序列严格递增，同一个数不可能在同一组合里出现两次，标记数组可以省去，代码更短也更快。

输出量提醒

n 取 20、r 取 10 时要输出十八万多行。评测时用 printf 输出，避免逐字符刷新拖慢速度。

D 题 · 读懂题面

P1605

迷宫

DFS · 回溯

N 行 M 列的网格里有 T 个障碍。从起点走到终点，每步上下左右移动一格，每个方格最多经过一次。问共有多少种不同走法。

输入格式

第一行 N、M、T：行数、列数、障碍数。

第二行四个数：起点 SX SY 与终点 FX FY。

接下来 T 行，每行一个障碍坐标。

输出格式

一个整数，即从起点到终点的方案总数。

只要经过的顺序不同，就算不同方案。

数据范围

N、M ≤ 5，T ≤ 10。

全图最多 25 格，DFS 暴力即可。

★ 保证起点上没有障碍。

样例

```
2 2 1
1 1 2 2
1 2
```

```
1
```

在 2×2 的迷宫里，(1,2) 是障碍。

从 (1,1) 只能走 (2,1)，再到 (2,2)，唯一一条路。

若没有障碍，走法会有 2 种：右下 或 下右。

题意建模的一句话

这是回溯搜索的最小完整模型：走进一格就标记，四方向试探，递归返回后立刻取消标记，让其他路径还能经过这一格。

D 题 · 思路拆解

信息学竞赛培训 · 洛谷六题习题解析

P1605 迷宫

数方案数与「能不能走到」不同：必须把每条路都走一遍再原样退回。回溯的两句口诀——走时标记、回来还原——就是全部核心。

第一步：地图与标记合一

g 数组用三个值区分状态：0 可走、1 障碍、2 当前路径已占。一个数组同时当地图和 visited 用，判断只有一句。

第二步：四方向数组

dx、dy 两个数组把上下左右打包成一个循环，免去四段重复代码。越界判断写在访问数组之前，防止读越界。

第三步：到达终点计数

进入 dfs 先判是否终点，是则方案数加一并返回。终点格无需再向四方向扩展，直接返回即可。

第四步：回溯还原现场

递归调用返回后立刻把 g 还原为 0。漏掉这一步，走过的格子永远封死，方案数会明显偏少。

最经典的错误

忘记回溯（还原 g）是本题半数错误答案的来源：程序不报错，但输出的方案数比正确值小。调试时可以先拿无障碍小迷宫验证。

D 题 · 参考代码

信息学竞赛培训 · 洛谷六题习题解析

P1605 · 回溯计数

```cpp
#include <cstdio>
using namespace std;

int n, m, t, fx, fy, cnt = 0;
int g[8][8];                    // 0 可走 1 障碍 2 已走过
int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};

void dfs(int x, int y) {
    if (x == fx && y == fy) {      // 到达终点，多了一种方案
        cnt++;
        return;
    }
    for (int k = 0; k < 4; k++) {
        int nx = x + dx[k], ny = y + dy[k];
        if (nx < 1 || ny < 1 || nx > n || ny > m) continue;
        if (g[nx][ny] != 0) continue;   // 障碍或已走过
        g[nx][ny] = 2;              // 标记：走进来
        dfs(nx, ny);
        g[nx][ny] = 0;              // ★ 回溯：还原现场
    }
}

int main() {
    scanf("%d %d %d", &n, &m, &t);
    int sx, sy;
    scanf("%d %d %d %d", &sx, &sy, &fx, &fy);
    for (int i = 0; i < t; i++) {
        int x, y;
        scanf("%d %d", &x, &y);
        g[x][y] = 1;
    }
    g[sx][sy] = 2;                  // 起点也要先标记
    dfs(sx, sy);
    printf("%d
", cnt);
    return 0;
}
```

先判终点再试探

到达终点立刻计数返回。若把终点当普通格子继续四方向搜索，会重复计数且多走无用路。

数组开大两圈

g 开 8×8 而 n、m 最大 5：配合下界 1 的判断，越界检查就不会漏。开得刚好反而容易在边界踩线。

起点标记别忘

起点也算「已经过」，必须先置 2 再搜索。漏标会导致某条路绕回起点被重复统计。

复杂度

25 格、每格 4 个方向，搜索树再大也在几十万次调用内，纯暴力即可通过，无需任何剪枝。

E 题 · 读懂题面

P1010

幂次方

递归 · 分治

任何正整数都能写成 2 的各次幂之和，且指数也要用同样的方式表示：a 的 b 次方写作 a(b)。按约定把 n 展开输出。

输入格式

一行一个正整数 n。

$1 \le n \le 2 \times 10^4$。

输出格式

一行字符串：n 的 0,2 表示。

指数继续按同规则展开；

表示中不能有空格。

数据范围

n 最大 20000，最高次幂 14。

递归深度极浅，直接写即可。

★ 指数为 1 时直接写 2，不加括号。

样例

```
1315
```

```
2(2(2+2(0))+2)+2(2(2+2(0)))+2(2(2)+2(0))+2+2(0)
```

1315 = 2 的 10 次方 + 2 的 8 次方 + 2 的 5 次方 + 2 + 1。

指数 10 本身又要展开成 2(2+2(0))+2，层层向内。

最后两项指数是 1 和 0：分别写 2 与 2(0)。

题意建模的一句话

把 n 写成二进制，每个 1 的位对应一项 2 的幂；指数若大于 2，就把它当成新的 n 继续展开——结构与原问题一模一样，递归呼之欲出。

E 题 · 思路拆解

这道题的递归结构在题面里已经写明白了，要做的是把「三项约定」精确翻译成输出逻辑，一个括号都不能错。

第一步：找最高次幂

对当前 n，反复除以 2 找到最高次幂 p。比如 n=137 时 p=7，这一项是 2 的 7 次方，剩余部分交给后面的加号。

第二步：三项输出约定

指数为 0：输出 2(0)；指数为 1：只输出 2，不加括号；指数大于等于 2：输出 2(指数的展开)，括号里递归。

第三步：处理剩余位

n 减去 2 的 p 次方后若还有剩余，先输出加号，再对剩余数递归调用自身，输出紧随其后。

第四步：递归出口与顺序

出口在 n 为 0 时输出 2(0)。整串按幂从高到低输出，与二进制从高位扫到低位一致，天然符合题面示例。

本题的失分点分布

绝大多数错误集中在两处：指数为 1 时多打了括号（写成 2(1)），以及剩余位为 0 时多打了加号。写完先手算 137 对照题面。

E 题 · 参考代码


```cpp
#include <cstdio>
using namespace std;

void solve(int n) {              // 输出 n 的 0,2 表示
    if (n == 0) {                // 递归出口：2 的 0 次方
        printf("2(0)");
        return;
    }
    int p = 0, t = n;
    while (t > 1) { t /= 2; p++; }  // 找最高次幂 p
    printf("2");
    if (p != 1) {                // 指数为 1 时只写 2
        printf("(");
        solve(p);                // ★ 指数也要继续展开
        printf(")");
    }
    if (n - (1 << p) > 0) {      // 还有更低的位
        printf("+");
        solve(n - (1 << p));
    }
}

int main() {
    int n;
    scanf("%d", &n);
    solve(n);
    printf("
");
    return 0;
}
```

找最高幂用循环除 2

t 反复除 2 计数即得 p，不用浮点 log，也没有精度问题。n 最大两万，p 不超过 14，循环十几次就结束。

指数为 1 的特判

p 等于 1 时跳过括号直接输出 2。这一行判断是全题最容易写漏的约定，漏掉后 2(1) 会铺满整份输出。

剩余位用减法传递

n 减去 2 的 p 次方就是剩下的数，继续递归。用左移计算幂，整数运算无误差，比 pow 稳。

与 F 题的呼应

本题把指数按二进制位逐层展开；下一题快速幂把指数按二进制位拆开相乘——「对指数做二进制分解」是同一思想的两种用法。

F 题 · 读懂题面

P1226

快速幂

分治 · 数学

给出三个整数 a、b、p，求 a 的 b 次方对 p 取模的值。指数与底数都可达 21 亿量级，朴素连乘必然超时，必须用快速幂。

输入格式

一行三个整数 a、b、p。

含义：求 a 的 b 次方 mod p。

输出格式

一行字符串，格式固定：

a^b mod p=s

其中 s 为运算结果，格式一字不差。

数据范围

a、b 不超过 2 的 31 次方量级。

p ≥ 2（不必担心模 1 的退化）。

★ 中间乘法必须用 long long。

样例

```
2 10 9
```

```
2^10 mod 9=7
```

2 的 10 次方 = 1024，1024 除以 9 余 7，所以输出如右。

若把 1024 完整算出再取模，本题侥幸能过；

但数据放大后连中间结果都存不下，必须边乘边取模。

题意建模的一句话

这是分治思想在乘法上的应用：把「算 b 次乘法」压缩成「算对数次平方」，指数每对折一次，问题规模就减半。

F 题 · 思路拆解

快速幂的全部逻辑四句话讲完：指数写成二进制、从低位往高位扫、当前位为 1 就乘进答案、底数每轮自乘一次。

第一步：把指数按二进制拆开

b 的二进制位决定了要乘哪些项：b=10 时二进制为 1010，即 8+2，所以 a 的 10 次方 = a 的 8 次方乘 a 的 2 次方。

第二步：维护自乘的底数

变量 a 每轮自乘一次，依次变成 a 的 1、2、4、8 次方……恰好对应二进制的每一位权重，最多自乘 31 次。

第三步：当前位为 1 就乘进答案

用 b 与 1 判断最低位；为 1 时把此刻的底数乘进 ans。扫完所有位，ans 就是 a 的 b 次方的完整拆解乘积。

第四步：每乘一步立刻取模

模的性质保证 (x·y) mod p 等于 (x mod p)·(y mod p) 再 mod p。每步乘完就取模，数值永远在 long long 范围内。

为什么不能先算完再取模

两个不超过 2 的 31 次方的数相乘，结果可达 2 的 62 次方，long long 恰好装得下；但三个连乘就溢出。所以规则是：每一步乘法之后马上取模。

F 题 · 参考代码


```cpp
#include <cstdio>
using namespace std;
typedef long long ll;

ll qpow(ll a, ll b, ll p) {        // 求 a 的 b 次方模 p
    ll ans = 1;                    // b 为 0 时循环不执行，ans=1
    while (b > 0) {
        if (b & 1) ans = ans * a % p;  // 这一位是 1：乘进答案
        a = a * a % p;             // 底数自乘，准备下一位
        b >>= 1;                   // 指数右移一位
    }
    return ans;
}

int main() {
    ll a, b, p;
    scanf("%lld %lld %lld", &a, &b, &p);
    printf("%lld^%lld mod %lld=%lld",
           a, b, p, qpow(a % p, b, p));  // ★ 底数先取模
    return 0;
}
```

位运算三件套

b 与 1 取最低位、b 右移一位、a 自乘——三句合成循环体。写成 b % 2 与 b / 2 也对，位运算写法是竞赛惯例。

ans 初值 1 的深意

任何数的 0 次方是 1，b 为 0 时循环一次都不执行，返回的正是 1，边界自动正确，无需特判。

输出格式一字不差

题目要求原样输出 a^b mod p=s，包括尖号与空格。格式题里格式分与算法分同样值钱。

循环次数

b 最多 31 个二进制位，循环恰好执行 31 次、每次两次乘法。对比朴素的 21 亿次乘法，快了三千万倍。

专题 · 快速幂（一）

F 题单独拿出三页细讲，因为快速幂是竞赛里出场率最高的模板之一。第一页先看清：朴素做法到底慢在哪里。

朴素连乘：a 乘 b 次	约 b 次	约 21 亿次乘法	必然超时
分治平方：每次对折指数	约 $2\log_2 b$ 次	约 60 次乘法	轻松通过
二进制快速幂：按位拆指数	约 $\log_2 b$ 次乘 + 移位	约 31 次乘法	模板级写法

> **分治的直觉：平方可以省一半**
>
> #0E7490

一句话记住复杂度

快速幂把乘法次数从「指数本身」降到「指数的二进制位数」，21 亿与 31 的差距，就是能不能通过的区别。

专题 · 快速幂（二）

用 a=2、b=10、p=9 走一遍模板循环。10 的二进制是 1010，共四轮；看 ans 与 a 如何交替变化。

每轮只有两个动作

指数位是 1 就把底数乘进 ans；无论哪一轮，底数都要自乘一次为下一位做准备。两句话构成循环主体。

取模发生在每次乘号之后

16 对 9 取模得 7，49 对 9 取模得 4——例子中的箭头就是取模。数值被死死按在 p 的量级内。

移位即读位

b 右移一位就丢掉已处理的最低位，循环条件 b 大于 0 保证每个二进制位恰好处理一次。

演算的价值

把样例逐轮手算一遍，比背十遍模板更牢。考场上忘了细节，就在草稿纸上按这张表演算小数据，循环怎么写立刻浮出来。

专题 · 快速幂（三）

什么时候想起快速幂、写的时候避开哪些坑——这页当作 checklist 使用，交题前逐条对照。

01

适用信号：题面出现「对 p 取模」

只要看到输出要求带 mod p，且指数或底数达到 10^9 量级，第一反应就应该是快速幂，而不是循环连乘。

02

底数先取模再进函数

a 本身可达 21 亿，先对 p 取模能保证第一次乘法也在安全范围。p 恒大于等于 2 时不必考虑模 1 特例。

03

乘法变量一律 long long

两个 31 位数相乘逼近 2 的 62 次方，只有 long long 装得下。函数参数、ans、底数三处都要声明为长整型。

04

指数为 0 自动正确

循环一次不执行，ans 初值 1 就是答案。任何数的 0 次方为 1，模板天然覆盖这个边界。

05

两个常用变式

矩阵快速幂：把整数乘法换成矩阵乘法，用于加速递推；龟速乘（快速乘）：把乘法换成加法，防止更大的数相乘溢出。

六题总结：一张表带回去

信息学竞赛培训 · 洛谷六题习题解析

考点 · 写法 · 坑点

把六道题的核心信息压成一张表。复习时先看「关键写法」一列，能默写出来就说明这一题真的掌握了。

P1025	**数的划分**	DFS · 剪枝	每份不小于前一份；剪枝剩余和	起点从 last 起才去重
P1928	**外星密码**	递归 · 模拟	遇 [读次数再递归，重复拼接	次数可能是两位数
P1157	**组合的输出**	DFS · 枚举	下一位从上一个数加 1 起	场宽 3 位不能少
P1605	**迷宫**	DFS · 回溯	走时标记、回来还原	忘回溯方案数变少
P1010	**幂次方**	递归 · 分治	指数也要展开；指数 1 只写 2	多打括号或多打加号
P1226	**快速幂**	分治 · 二进制	按位拆指数，每乘一步取模	中间乘法溢出 int

一句话带走

搜索看「起点约束」：相等从 last 起、互异从 last 加 1 起；回溯看「还原现场」；递归看「出口与输出时机」；快速幂看「按位拆指数、边乘边取模」。

会写模板，更要看清结构

六道题，三条主线：起点约束的搜索、逐层展开的递归、按位拆分的快速幂

课后任务 · 每题手敲一遍并提交，重点对照「最易错的一点」自查

下一步方向 · 广度优先搜索 · 记忆化 · 矩阵快速幂 · 逆元与取模运算

洛谷六题习题解析 · 完